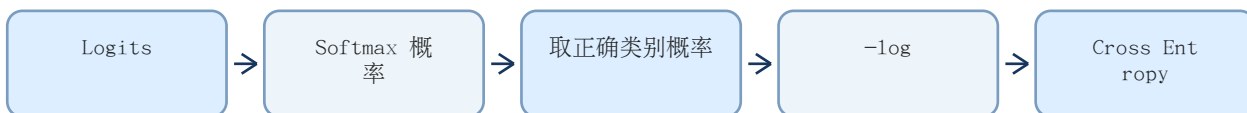


第18讲：Cross Entropy

从正确答案概率到 Next-token Prediction Loss

核心概念：【必须真正掌握】

本讲不是让你背公式，而是让你能从 logits 亲手算出概率、正确类别概率、Cross Entropy，并能把这一过程重新放回 LLM 的 next-token prediction。



建议学习顺序：先看讲义 → 手算示例 → 运行实验代码 → 做练习 → 用速查卡复盘。

一、本讲学习目标

能够解释 Cross Entropy 在分类任务里究竟惩罚什么，而不是只会背 $-\log(p)$ 。

能够从 logits $\rightarrow$ Softmax $\rightarrow$ p_correct $\rightarrow$ Cross Entropy 手算一个三分类例子。

能够解释 one-hot 公式为什么在单标签分类时退化成 $-\log(p_{\text{correct}})$ 。

能够读懂 Batch logits 的 Shape，并理解 reduction="none" / "mean"。

知道 PyTorch CrossEntropyLoss / F.cross_entropy 应直接接收 logits。

能够把 Cross Entropy 放回 LLM 的 (B,T,V) 输出和 next-token target shift。

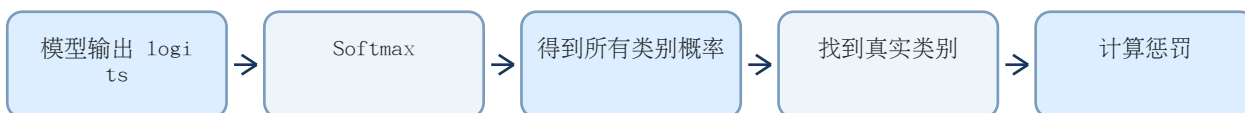
知道 loss 较低表示训练目标更匹配，但不等价于事实正确、校准良好或下游 Agent 更可靠。

核心概念：和上一讲的连接

第17讲解决了“logits 怎样变成概率”。本讲只再做两步：找到真正正确类别的概率，然后用 $-\log$ 把它变成一个可优化的标量损失。

二、【原理】Cross Entropy 到底是什么？

在最常见的单标签分类里，模型会为每个候选类别产生一个 `logit`。Softmax 把这些 `logits` 变成一组总和为 1 的概率。训练时我们已经知道真实答案是哪一类，因此只需要问：模型给真实正确类别分了多少概率？



$$L = -\log(p_{\text{correct}})$$

如果模型把正确答案概率分得很高，`p_correct` 接近 1， $-\log(p_{\text{correct}})$ 就接近 0；如果模型对正确答案非常不自信，`p_correct` 很小，`loss` 就会很大。

数学直觉：Loss 的方向

训练优化器最终会努力降低这个标量。对 Cross Entropy 来说，降低 `loss` 的主要方式之一，就是提高真实正确类别相对于其他类别的 `logit`。

【通俗解释】把概率理解成“押注”

假设一道三选一题的正确答案是 A。模型不是只给一个答案，而是分配一组“押注比例”：A 65.9%，B 24.2%，C 9.9%。Cross Entropy 并不关心模型把 B 和 C 谁排第二，它首先看“正确答案 A 到底拿到了多少概率”。

数学直觉：为什么不是直接用 $1-p_{\text{correct}}$ ？

$-\log(p)$ 在概率接近 0 时惩罚会急剧变大。也就是说，模型“稍微不确定”和“非常自信地把正确答案判成几乎不可能”会受到明显不同的惩罚。

三、【具体数字例子】从 logits 手算一次 Cross Entropy

设三分类模型输出：

$$\text{logits} = [2.0, 1.0, 0.1]$$

真实类别为第 0 类。上一讲已经算出：

$$\text{Softmax}(\text{logits}) = [0.659001, 0.242433, 0.098566]$$

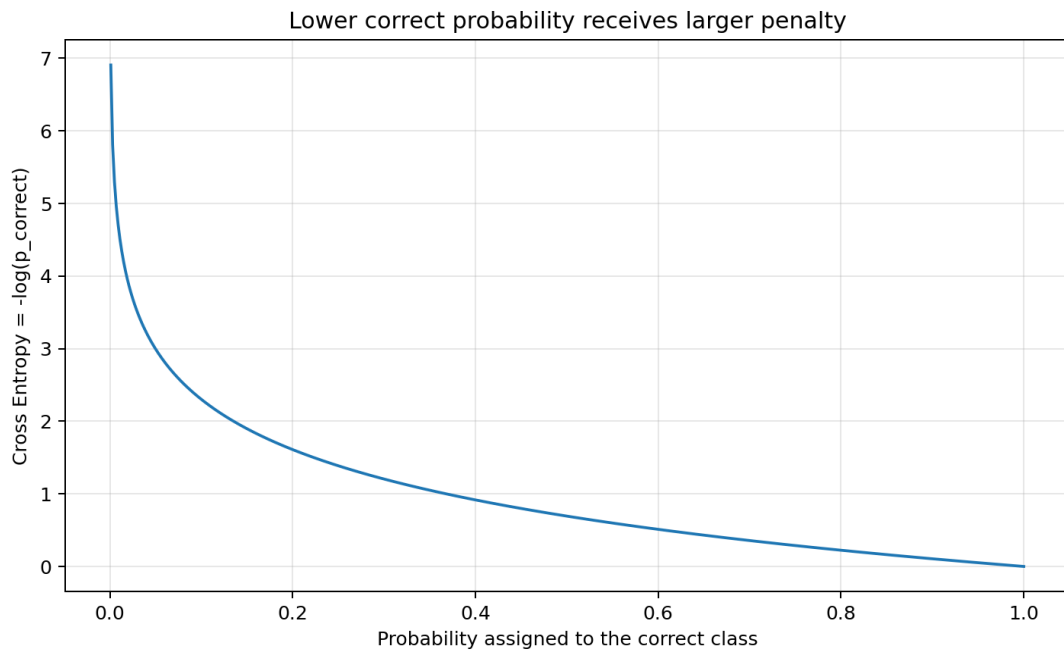
因此正确类别的概率：

$$p_{\text{correct}} = 0.659001$$

Cross Entropy:

$$L = -\log(0.659001) = 0.417030$$

真实类别	p_correct	Cross Entropy	直觉
0	0.659001	0.417030	模型最偏向真实类别，loss 最小
1	0.242433	1.417030	真实类别只排第二，惩罚更大
2	0.098566	2.317030	真实类别概率最低，惩罚最大



图中横轴越靠近 0，代表模型给正确答案的概率越低；纵轴 $-\log(p)$ 会快速升高。

四、【必要公式】完整 Cross Entropy 公式怎么读？

$$H(y, p) = - \sum_i y_i \log(p_i)$$

逐项拆解：

符号	含义	在三分类中的例子
i	类别索引	0, 1, 2
y_i	真实标签分布	正确类别为 1，其余为 0
p_i	模型对第 i 类的预测概率	Softmax 输出
$\log(p_i)$	该概率的对数	概率越小，对数越负
负号 $-$	把负的 $\log$ probability 变成正惩罚	正确概率越小，loss 越大

如果真实标签是 one-hot:

$$y = [1, 0, 0]$$

$$H = -(1 \cdot \log(p_0) + 0 \cdot \log(p_1) + 0 \cdot \log(p_2)) = -\log(p_0)$$

核心概念：为什么代码常只传 target index？

因为单标签分类中 one-hot 的绝大多数位置都是 0。框架只需要知道正确类别索引，就能直接取对应的 $\log$ probability，不必真的构造完整 one-hot 向量。

五、Batch Cross Entropy：Shape 到底是什么？

假设一个 Batch 有 B 个样本，每个样本有 C 个候选类别：

`logits.shape = (B, C)`

`targets.shape = (B,)`

每一行 `logits` 都会在类别维度 C 上形成一个概率分布；`targets` 中第 b 个整数指出第 b 个样本的正确类别。

对象	Shape	每个数字代表什么
<code>logits</code>	(B, C)	第 b 个样本对第 c 个类别的未归一化分数
<code>probabilities</code>	(B, C)	每个样本在 C 个类别上的 Softmax 概率
<code>targets</code>	$(B,)$	每个样本的正确类别索引
<code>per-sample loss</code>	$(B,)$	每个样本自己的 $-\log(p_{\text{correct}})$
<code>mean loss</code>	scalar	把 B 个样本 <code>loss</code> 求平均后的训练标量

本讲实验中的 3 个样本得到：

`per_sample_loss = [0.185182, 0.263733, 0.453090]`

`mean_loss = 0.300668`

科研连接：为什么论文里常报告 `mean loss`？

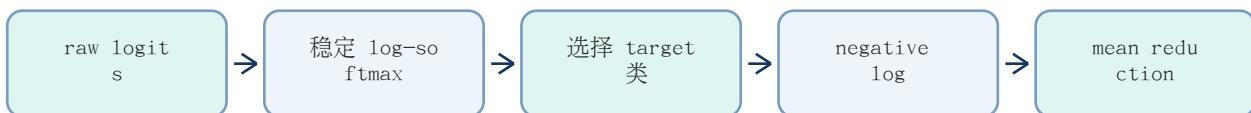
`mean reduction` 让 `loss` 的尺度不会仅因为 `Batch Size` 线性翻倍，更便于在不同 `Batch` 下解释；但梯度尺度与具体框架、`reduction`、分布式聚合方式仍要结合实现判断。

六、PyTorch 里 Cross Entropy 到底帮你做了什么？

常见写法：

```
# logits: [B, C]
# target: [B]
loss = torch.nn.functional.cross_entropy(logits, target)
```

关键点：这里直接传 logits，不要先手工 Softmax。从数学上可以理解为框架内部稳定地完成了 log-softmax，再取真实类别对应的 negative log likelihood。



常见误区 / 风险：常见错误：先 softmax 再 cross_entropy

F.cross_entropy 期望的是未归一化 logits。把 probabilities 再当 logits 输入，相当于对“已经归一化过的一组数”重新做 log-softmax。它可能不报错，但算的已经不是原来想要的 Cross Entropy。

本讲实际实验：

```
print(F.cross_entropy(logits, target) = 0.300668
```

```
print(F.cross_entropy(softmax(logits), target) = 0.938085
```

七、数值稳定性：为什么常把 Softmax 和 Log 合在一起？

如果直接对很大的 logits 做 exp, 例如:

[1000, 1001, 999]

计算机会先遇到指数溢出, 得到 inf; inf / inf 随后变成 nan。

naive exp = [inf, inf, inf]

naive probability = [nan, nan, nan]

稳定实现会利用“整体平移不改变 Softmax”的性质, 先减去最大值:

[1000, 1001, 999] - 1001 = [-1, 0, -2]

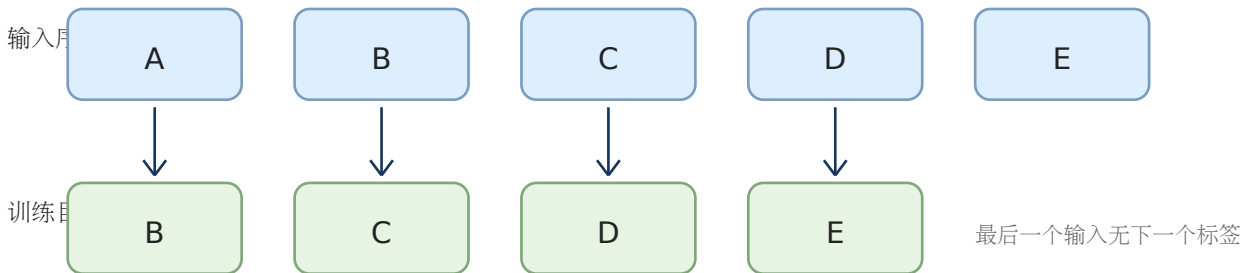
然后再计算概率和 log。框架里的 Cross Entropy 通常还会进一步用 log-sum-exp 形式避免不必要的中间数值风险。

科研连接：科研代码审查点

如果论文或自己写的代码手工 `exp -> sum -> log`, 尤其在 float16/bfloat16 或大 logits 下, 要检查是否使用稳定写法。数值稳定不是“工程小问题”, 它会直接影响训练是否出现 NaN。

八、把 Cross Entropy 接回 LLM: Next-token Prediction

语言模型训练不是“一整句只算一个分类”。它通常在每个有效 token 位置都做一次“从整个词表中预测下一个 token”的多分类。



玩具序列: A -> B -> C -> D -> E。训练关系是:

当前位置输入	要预测的下一个 token	这是一个什么问题
A	B	从整个词表 V 中分类选择 B
B	C	从整个词表 V 中分类选择 C
C	D	从整个词表 V 中分类选择 D
D	E	从整个词表 V 中分类选择 E

$\text{logits.shape} = (B, T, V)$

$\text{targets.shape} = (B, T)$

其中 V 是 vocabulary size。对每个 (b, t) 位置, $\text{logits}[b, t, :]$ 都是一整个词表上的候选分数。

核心概念: 你以前在 LLM 课程里见过它

以前学 Pretraining / Next-token Prediction 时, 你看到的是“预测下一个 Token”。现在从底层看, 它实际上就是: 每个 token 位置产生 V 个 logits -> Cross Entropy -> 对有效 token 的 loss 做聚合 -> Backpropagation。

九、LLM 代码里为什么经常 reshape / flatten?

框架的普通 Cross Entropy 常用二维形式 (N,C)。因此 LLM 经常把:

$(B, T, V) \rightarrow (B \cdot T, V)$

$(B, T) \rightarrow (B \cdot T,)$

把所有 token 位置暂时视为 $B \cdot T$ 个分类样本。分类类别数仍然是 V 。

```
# logits: [B, T, V]
# labels: [B, T]
flat_logits = logits.reshape(B * T, V)
flat_labels = labels.reshape(B * T)
loss = F.cross_entropy(flat_logits, flat_labels)
```

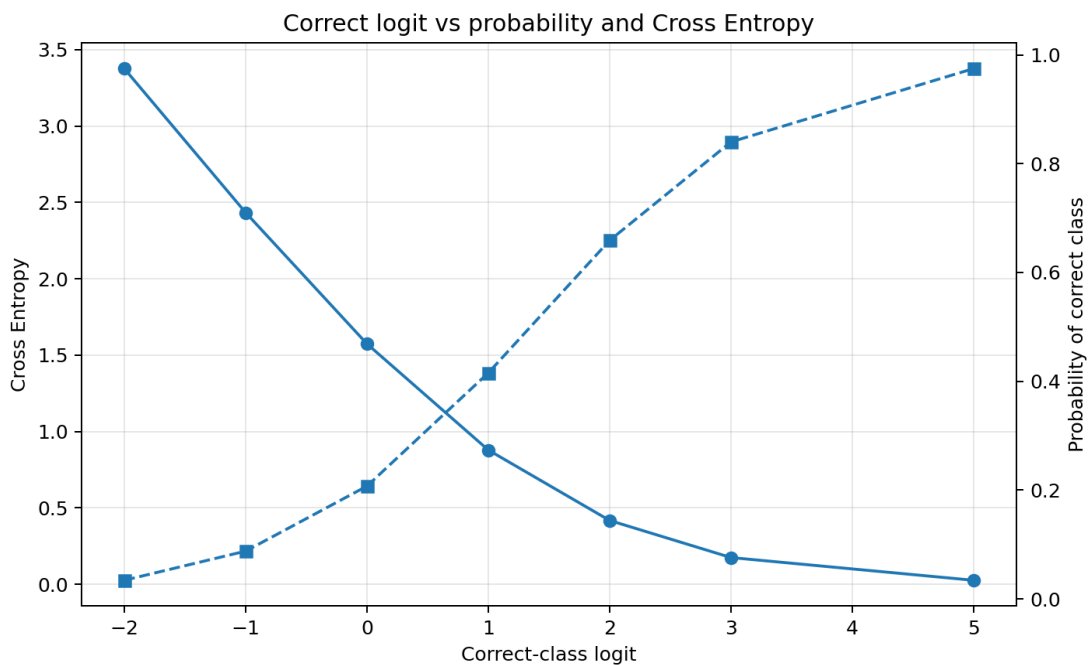
核心概念: Shape 判断

展平的是 batch 轴和 sequence 轴; vocabulary 轴必须保留下来作为“类别维”。如果把 V 也一起 reshape 掉, Cross Entropy 就不知道哪些数字在竞争同一个 target。

实际 LLM 实现还会处理 padding、被屏蔽 token 或“不参与训练”的位置, 常见做法是 mask 或 ignore_index。这一讲只需要理解: 最终 loss 应该只在有效训练位置上聚合。

十、【代码实验】本讲实验到底验证什么？

实验	问题	验证结论
1	Softmax 后怎样得到 CE?	$CE = -\log(p_{\text{correct}})$
2	同 logits 不同 target 呢?	target 决定取哪一个概率计算惩罚
3	提高正确类 logit 会怎样?	p_{correct} 上升, CE 下降
4	one-hot 公式和 index 公式一致吗?	完全等价
5	Batch 怎么聚合?	先得到 per-sample loss, 再做 mean
6	大 logits 会不会溢出?	naive 会, 稳定写法不会
7	PyTorch 是否和 NumPy 一致?	最大误差约 $5.55e-17$
8	LLM token loss 怎么来?	每个位置都有自己的 next-token CE
9	(B,T,V) 为什么能 flatten?	把 $B \times T$ 个 token 当分类样本, V 保持为类别维



正确类别 logit 提高时, $p(\text{correct})$ 上升, 而 Cross Entropy 单调下降。

十一、【常见误区】

常见误区 / 风险：误区1：Cross Entropy 越低，模型就一定更“聪明”

CE 只直接衡量训练目标上的预测分布与标签之间的匹配程度。事实性、鲁棒性、校准、推理可靠性、Agent 工具调用正确率，都需要额外评测。

常见误区 / 风险：误区2：Softmax 概率就是现实世界的真实概率

Softmax 产生数学上的归一化分布，但模型可能过度自信。Calibration 是另一个独立问题。

常见误区 / 风险：误区3：target 是 one-hot，就必须手工构造巨大 one-hot 向量

PyTorch 单标签 Cross Entropy 通常直接使用类别索引，更高效。

常见误区 / 风险：误区4：先 Softmax 再传给 CrossEntropyLoss 更保险

恰恰相反。标准 PyTorch 用法是直接给 logits。

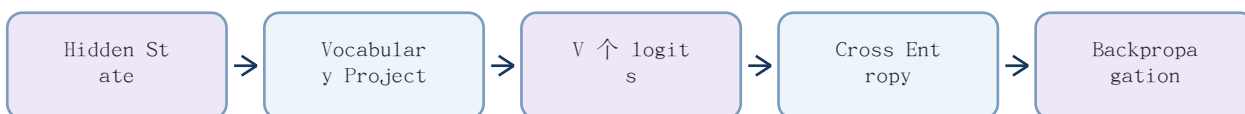
常见误区 / 风险：误区5：LLM 一条序列只产生一个 Cross Entropy

典型 causal LM 会在很多 token 位置分别产生 token loss，然后对有效位置聚合。

常见误区 / 风险：误区6：Loss 降了就说明测试性能一定提升

训练 loss、验证 loss、下游任务指标之间可能出现分离。后续 Generalization 阶段会专门讨论。

十二、【与 LLM / Transformer / Agent Systems 的关系】



在 GPT 类语言模型中，Transformer Block 先为每个位置产生 hidden state；最后通过 vocabulary projection 得到对整个词表的 logits；再用 shifted labels 计算 next-token Cross Entropy。

科研连接：与 Transformer 的关系

Cross Entropy 本身不属于 Attention 或 Transformer Block 内部结构；它是训练目标的一部分。模型架构决定“怎样产生 logits”，Loss 决定“什么样的 logits 更好”。

科研连接：与 Agent Systems 的关系

Agent 往往直接调用已经训练好的 LLM，因此你不一定天天自己算 CE；但理解它很重要，因为它决定基础模型是怎样学会 token prediction 的，也帮助你判断 fine-tuning、SFT、LoRA 论文里的训练设置是否合理。

科研连接：科研阅读提醒

论文若只报告更低 language modeling loss，不能自动推出 Agent 成功率、长程规划、工具可靠性或安全性更高。需要看它是否给出了与你研究问题一致的下游指标与对照。

十三、本讲最低验收标准

给你一组小 logits 和 target, 你能手算 Softmax、p_correct 和 Cross Entropy。

你能解释为什么 p_correct 越小, $-\log(p_{\text{correct}})$ 越大。

你能解释 one-hot CE 为什么等于 $-\log(p_{\text{correct}})$ 。

你看到 `logits.shape=(B,C)` / `target.shape=(B,)` 能说清每个维度。

你知道 PyTorch `F.cross_entropy` 直接接 logits, 不应先做 softmax。

你看到 LLM `logits.shape=(B,T,V)` 时, 知道 V 是分类类别维。

你能解释 next-token labels 为什么相对输入向左错一位。

你知道 CE 较低并不等价于 Agent 系统整体更可靠。

核心概念: 下一阶段

至此第13-18讲完成了“导数最小基础 + Loss + MSE + 概率/指数/对数 + Softmax + Cross Entropy”。下一讲开始正式进入 Gradient: 多个参数同时影响 Loss 时, 怎样把每个参数的局部敏感度组织起来。